

SYSTEM BLUEPRINT

TETHERLESS: Decentralized E2EE Messaging

Executive Overview & System Architecture

The Zero-Trust Relay Axiom

Active Adversary Model: The relay server is assumed compromised, malicious, or state-monitored. It will inspect, alter, drop, or replay every byte it can.

Zero Plaintext Knowledge: The relay holds zero cryptographic keys and never sees plaintext. Message payload bytes are strictly opaque ciphertext.

Reply Metadata Hidden: Quotes and conversation context travel inside the ciphertext envelope, hiding communication sub-graphs from intermediate nodes.

Dual-Layer Protection: TLS 1.3 is applied in addition to E2EE for transport integrity and traffic hiding, never as a substitute for E2EE.

Theoretical Foundation: Adheres to the formal secure messaging taxonomy of Unger et al. [\[Unger et al., IEEE S&P 2015\]](#)

Core Architectural Invariants

End-to-End Privacy: Messages encrypted on sender device and decrypted only on receiver device. Mathematical confidentiality guaranteed by AES-256-GCM.

Cryptographic Identities: Addresses derived from identity public key hashes, preventing display name spoofing or account displacement.

Tamper-Evidence: All routed frames cryptographically signed with RSA-2048 identity keys over deterministic byte encodings.

Memory Isolation: Session keys derived via HKDF-SHA256 exist in transient memory only; keys are zeroed and discarded on restart.

OTR Lineage: Follows early Off-the-Record paradigms establishing ephemeral keys over untrusted networks. [\[Borisov et al., WPES 2004\]](#)

MODULE TOPOLOGY

Multi-Module Codebase Architecture

Four decoupled Gradle modules separated by rigid platform and trust boundaries

core-shared (Platform-Neutral Core)

Cryptographic primitives (AES/DH/RSA), wire codec, session state machine, HKDF. Emits Java 8 bytecode for Android compatibility and Java 11+ for desktop.

chat-server (Untrusted Dumb Relay)

High-throughput ciphertext forwarder. TLS 1.3 server, non-blocking queue routing, token-bucket rate limiting, loopback Prometheus metrics. Holds zero keys.

chat-desktop (Java Swing Client)

Swing desktop UI, background worker thread pool, SQLite in WAL mode, PBKDF2 column encryption, and jpackage Windows .msi bundling.

chat-mobile (Android Client)

Modern Jetpack stack: Room ORM, SQLCipher whole-file encryption, AndroidKeyStore hardware keys, and persistent foreground socket service.

WIRE CODEC & SERIALIZATION

Binary Wire Framing vs. Java Serialization

Eliminating remote code execution (RCE) and memory leaks at the network boundary

The Java Deserialization Hazard (Legacy)

RCE Vulnerability: `ObjectInputStream.readObject()` executes arbitrary gadget chains during instantiation, exposing clients and relay to RCE. [\[Frohoff & Lawrence, AppSecCali 2015\]](#)

Vulnerability Taxonomy: Classified as CWE-502 (Deserialization of Untrusted Data). [\[CWE-502\]](#)

Memory Bloat Risk: `ObjectOutputStream` maintains internal object back-reference tables, causing memory leaks and eventual `OutOfMemoryError` crashes.

Allocation Bomb: Malformed frame length headers could trigger multi-gigabyte array allocations, causing immediate remote Denial of Service.

The Tetherless Binary Frame Codec

Framing Format: [4-byte length] [1-byte version]
[wireCode: int] [msgId: UTF] [sender: UTF] [receiver: UTF] [ts: long] [iv] [payload] [signature]

Pre-Allocation Caps: IDs ≤ 128B, Payload ≤ 64KiB, Signature ≤ 512B, Frame ≤ 1MiB. Rejects oversized frames BEFORE memory allocation.

Zero Native Serialization: `Serializable` interface completely purged from repository. Codec uses explicit `DataInput/DataOutput` parsing with typed `ProtocolException`.

Fuzz-Tested Resilience: Subjected to 20,000 random bit mutations and fuzz tests; guarantees clean typed rejection without crashing.

CRYPTOGRAPHIC SPECIFICATION

Cryptographic Primitives & Specifications

Adherence to vetted standards, constant-time algorithms, and safe parameters

Cipher Suite & Key Agreement

Payload Encryption: AES-256-GCM with 96-bit nonce and 128-bit authentication tag. Provides confidentiality and authenticated integrity. [\[NIST SP 800-38D\]](#)

Diffie-Hellman Group: RFC 3526 MODP Group 14 (2048-bit safe prime, $g=2$). Replaces expensive safe-prime generation (<50ms vs ~30s). [\[RFC 3526\]](#)

Group Validation: Inbound DH public keys validated: rejected if $y \leq 1$, $y \geq p-1$, or $y^q \bmod p \neq 1$. Completely blocks small-subgroup attacks.

Foundational Theory: Diffie-Hellman public key cryptography foundation. [\[Diffie & Hellman, IEEE TIT 1976\]](#)

Signatures & Transport Protection

Identity & Signatures: RSA-2048 with SHA256withRSA. Signs handshakes and messages over canonical deterministically ordered byte representations.

Anti-Stripping Guard: The codec treats missing signatures on signed types as fatal protocol errors, preventing signature-stripping downgrade attacks.

At-Rest Password KDF: PBKDF2-HMAC-SHA256 at 210,000 iterations over random 16-byte salt for desktop database encryption key derivation. [\[NIST SP 800-132\]](#)

Transport Security: TLS 1.3 restricted to AEAD cipher suites with strict certificate pinning on both desktop and mobile clients. [\[RFC 8446\]](#)

NONCE CONSTRUCTION & GCM SAFETY

Deterministic GCM Nonce Construction

Eliminating catastrophic AES-GCM nonce reuse across communication directions

The AES-GCM Nonce Reuse Hazard

Catastrophic Failure: Reusing an IV/nonce under the same key in GCM immediately leaks the XOR of plaintexts and allows polynomial forgery of authentication tags. [\[McGrew & Viega, INDOCRYPT 2004\]](#)

Random IV Flaw: A random 96-bit IV has a non-negligible collision probability by the Birthday Paradox over millions of messages.

Bidirectional Collision: Early bug discovery: Alice and Bob both starting at counter 0 with same key collided immediately on message #1 across directions.

Tetherless Deterministic Nonce

Structure: 12-byte (96-bit) buffer: [Direction: 4 bytes]
|| [Monotonic Counter: 8 bytes]

Direction Derivation: Direction bit is deterministically computed by lexicographical sorting of Peer IDs: Initiator = 0x00000000, Responder = 0x00000001.

Send Budget Cap: Hard cap of 100,000 messages per session key. Attempting to exceed budget triggers mandatory rekeying, preventing counter wraparound.

Empirical Verification: Verified across 100,000 continuous message transmissions with zero duplicate nonces (lvReuseTest).

KEY DERIVATION & NORMALIZATION

HKDF-SHA256 Derivation & Normalization

Extract-then-Expand PRF and solving cross-platform leading zero byte bugs

HKDF-SHA256 Key Derivation

Extract Phase: $PRK = \text{HMAC-SHA256}(\text{salt} = \text{nonceA} \parallel \text{nonceB}, \text{IKM} = \text{DH_shared_secret})$. Extracts pseudorandom key from Diffie-Hellman secret.

Expand Phase: $OKM = \text{HMAC-SHA256}(PRK, \text{info} = \text{'tetherless-v1 aes-256-gcm'} \parallel 0x01, L = 32 \text{ bytes})$. Derives cryptographically isolated session key.

Formal Specification: Formal security analysis of Extract-then-Expand PRF paradigms. [\[Krawczyk, CRYPTO 2010 / RFC 5869\]](#)

Test Vector Verified: Verified against published RFC 5869 golden test vectors in CryptoVectorsTest.

Leading Zero Padding Normalization

The Intermittent Flaw: DH shared secrets can produce leading zero bytes (~1 in 256 handshakes). Conscript (Android) and SunJCE (desktop) disagree on stripping them.

Catastrophic Consequence: Without normalization, JVM and Android derived different keys on ~0.4% of sessions, causing silent handshake failures.

The Deterministic Fix: Left-pad the shared secret to the exact byte length of modulus p before feeding HKDF.

Empirical Proof: 1,000 consecutive handshakes derive matching keys with zero mismatches (CryptoVectorsTest).

FORMAL THREAT MATRIX

Threat Matrix: Attacks T1 through T6

Comprehensive mitigations and automated verification backing each security claim

T1: Passive Relay Sniffing

AES-256-GCM encryption with client-held keys. Verified by EndToEndExchangeTest with live wire tap asserting zero plaintext.

T2: Active Relay MITM

Handshake signed with long-term RSA key; Peer ID bound to hash of public key. Verified by AdversarialRelayTest MITM scenarios.

T3: Network Interception

Pinned TLS 1.3 transport. Protects metadata and connection integrity from intermediate ISPs and eavesdroppers.

T4: Replay & Reordering

Monotonic counter sliding window (floor = highest - 1024) + 5-min timestamp skew check. Replayed frames rejected silently.

T5: Malicious Peer Payload

Explicit length-prefixed binary codec with per-field size limits. Fuzz-tested with 20,000 mutations; no deserialization sinks.

T6: Device Compromise at Rest

SQLCipher full-database encryption on Android (AndroidKeyStore); PBKDF2 column encryption on Desktop. Keys never written in plaintext.

IDENTITY & TRUST ARCHITECTURE

Identity Architecture: TOFU & Safety Numbers

Binding identities cryptographically rather than relying on mutable display names

Cryptographic Peer Identity

Address Derivation: Peer ID = SHA-256(PublicKey)[0..16] (32 lowercase hex characters). Address never changes when a user changes display name.

Self-Signed HELLO Proof: Clients transmit public key and sign their HELLO frame. Relay and peers reject any client whose ID does not match key hash.

Display Name Sanitization: Display names are untrusted metadata. Stripped of bidirectional unicode overrides and control characters to prevent UI spoofing.

Formal Models: Multi-stage key exchange security. [\[Cohn-Gordon et al., EuroS&P 2017\]](#)

Trust-on-First-Use & Safety Numbers

TOFU Binding: First key seen for a peer is pinned. If a subsequent key differs, sending is blocked and the UI alerts KEY_CHANGED.

Safety Number Format: 12 groups of 5 decimal digits (60 digits total) displayed in UI.

Deterministic Symmetry: Computed over SHA-256(minKey || maxKey); identical on both ends regardless of who initiates.

Short Authenticated Strings: Implements Vaudenay's SAS authentication protocol for out-of-band human verification. [\[Vaudenay, CRYPTO 2005\]](#)

SYSTEM BOUNDARIES & NON-GOALS

Honest Security Limitations & Non-Goals

Explicitly documenting system boundaries and what Tetherless does NOT protect

Architectural Non-Guarantees

Relay Observes Metadata: The relay observes the social graph: who talks to whom, timestamps, frequency, and frame byte lengths.

Single Point of Failure: Centralized relay topology can deny service to clients. True federation is planned for post-1.0.

No Group Messaging: Protocol is strictly point-to-point. Decentralized group messaging requires MLS. [\[RFC 9420 \(MLS\)\]](#)

No Multi-Device Sync: One identity per device. A second device has a completely independent identity key.

Cryptographic Scope Boundaries

No Per-Message Ratchet: DH exchange establishes a session key for up to 100,000 messages. Compromise of a session key compromises that session.

Session Expiry by Volume: Key renewal is triggered by message count (100k budget), not by elapsed time TTL. In-memory keys cleared on restart.

Live Device Compromise: Files are encrypted at rest, but an adversary running code inside the live process memory can access decrypted messages.

Unaudited Status: While backed by rigorous automated test suites, Tetherless has not yet undergone third-party commercial security audits.

RELAY FORWARDING ARCHITECTURE

Dumb Relay: Non-Blocking Ciphertext Routing

High-throughput message routing with zero payload inspection

Non-Blocking Per-Client Writer Queues

Dedicated Writer Threads: Each client session maintains an `ArrayBlockingQueue<byte[]>` (capacity 256) serviced by a single dedicated writer thread.

No Head-of-Line Blocking: A slow recipient cannot block the sender's thread. Handlers merely enqueue frames and immediately return.

Overflow Protection: If a recipient queue fills beyond capacity, the connection is terminated cleanly with an ERROR frame rather than deadlocking.

Zero Decoding: Routes based on header fields without decoding payload bytes.

Zero-Knowledge Forwarding

Verbatim Forwarding: Relay forwards frames without re-encoding, preserving cryptographic signatures intact.

Redacted Logs: Logs peer interactions using SHA-256 truncated IDs (Redact.id), never real IDs or payload bytes.

Stress Verified: 10 concurrent senders x 1,000 messages (10,000 total) to one recipient delivered without loss or corruption.

CONNECTION CONCURRENCY & LIFECYCLE

Connection Lifecycle & Race Defenses

Atomic CAS teardown, half-open detection, and clean resource management

Atomic Reconnect (CAS Protection)

The Reconnect Race: A dying socket's cleanup thread racing a new incoming connection previously evicted the active client.

CAS Atomic Removal: `ClientRegistry.remove(clientId, expectedSession)` removes only if the mapped value matches this exact dying socket instance.

Duplicate ID Rejection: Two clients claiming the same ID are cleanly rejected.

Graceful Teardown: Clients send DISCONNECT on exit; server cleans registry and closes worker threads cleanly.

Heartbeats & Deadlock Prevention

Deadlock Detection: `SO_TIMEOUT` of 90s prevents orphaned sockets from holding handler threads indefinitely.

Heartbeat PING/PONG: Server issues PING every 30s; closes connection after 2 consecutive missed PONGs.

Shutdown Hook: Relay shutdown hook sends DISCONNECT to all peers and drains thread pool before JVM exit.

Leak-Free Proof: 1,000 rapid connect/disconnect cycles leave thread count at baseline (`ChatServerLifecycleTest`).

OPERATIONAL HARDENING & GATING

Operational Hardening & Release Gating

Eliminating default development keys and securing host environments

Release Gate Enforcement (-PreleaseBuild)

The Dev Certificate Trap: In dev, relay uses a known self-signed cert from repo scripts. Shipping with this would allow trivial MitM by anyone.

Refusal to Fall Back: Packaged relay strictly refuses to start unless a custom keystore is configured; development keystore is stripped from JAR and Docker image.

Secret Management: Prefers KEYSTORE_PASSWORD_FILE over environment variables to prevent leaking credentials in process lists or docker inspect.

Container & Host Sandboxing

Docker Compose Hardening: Runs unprivileged as non-root user (tetherless:tetherless), read-only root filesystem, drop ALL Linux capabilities.

Loopback Metrics: Prometheus metrics on PORT + 1 bound strictly to 127.0.0.1; unauthenticated telemetry is never reachable from WAN.

systemd Unit Hardening: Configured with ProtectSystem=strict, PrivateTmp=true, ProtectHome=true, and system call filtering.

CLIENT PERSISTENCE & PACKAGING

Client Implementations: Desktop & Mobile

Swing UI, SQLite WAL, Room, SQLCipher, and background services

Desktop Client (chat-desktop)

EDT Decoupling: All network I/O and cryptographic operations execute off Event Dispatch Thread. UI updates marshaled via `invokeLater`.

SQLite in WAL Mode: Configured with `PRAGMA journal_mode=WAL` and `busy_timeout=5000`; single-writer connection preventing database lock contention.

In-Process Keygen: Identity keys generated strictly in-process; eliminated legacy flaw that shelled out to `keytool` exposing passphrases in CLI args.

Self-Contained Packaging: Packaged via `jpackage` into a standalone `.msi` on Windows, embedding a stripped custom JRE runtime (users need no JDK).

Mobile Client (chat-mobile)

Jetpack Architecture: Built on Room ORM, ViewModel, LiveData reactive streams, and ListAdapter with DiffUtil.

SQLCipher Full-File Encryption: Unlike desktop column encryption, Android encrypts entire SQLite database file including tables, participants, and timestamps. [\[SQLCipher Design Spec\]](#)

Persistent ChatService: Configured as an Android Foreground Service with continuous notification; survives backgrounding, Doze mode, and memory trim.

Update Checker (CLIENT-DESKTOP-08): Async GitHub release polling with non-intrusive UI banner; no dangerous auto-execution of unsigned binaries.

ADVERSARIAL VERIFICATION SUITE

Adversarial Testing & Verification Suite

Proving security properties through deliberate mutation and attack injection

AdversarialRelayTest (13 Attack Scenarios)

Key Substitution MITM: Relay attempts to substitute DH public keys during handshake; clients abort handshake immediately.

Identity Forgery: Relay substitutes public key inside HELLO frame; rejected because Peer ID does not match public key hash.

Ciphertext Bit-Flipping: Single bit flipped in ciphertext payload; GCM authentication tag verification fails, message dropped with warning.

Replay Attack: Relay re-transmits valid captured frame; client detects duplicate counter within sliding window and drops frame.

[\[Perrig et al., TESLA 2002\]](#)

End-to-End & Golden Conformance

EndToEndExchangeTest: Transmits 200 real messages through live relay with external wire tap; mathematically asserts zero plaintext substrings in transit.

Cross-Platform Vectors: 17 frozen wire-format vectors (ProtocolVectors) tested across JVM and Android emulator, preventing cross-platform codec drift.

Zero-Warning Static Analysis: SpotBugs with find-sec-bugs plugin runs on all modules with ZERO unresolved high-severity vulnerabilities.

PRODUCTION ROADMAP & REFERENCES

MVP Delivery Roadmap & Academic Citations

Prioritized ticket backlog for production deployment and foundational literature

Prioritized MVP Ticket Backlog

- 1. CLIENT-DESKTOP-07 (Active):** Desktop test coverage: unit tests for MessageRepository and ConnectionManager.
- 2. INTEG-01 (High Priority):** End-to-end headless integration harness: automated multi-client exchange over live relay.
- 3. BUILD-04 (High Priority):** Continuous integration: GitHub Actions pipeline running clean build and SpotBugs.
- 4. REL-01 (High Priority):** Security review pass: formalize threat model, audit logs for zero secret leaks.
- 5. CLIENT-DESKTOP-08 (MVP Feature):** In-app update notifications: async GitHub release polling with non-intrusive banner.
- 6. REL-03 & INTEG-04 (Release Prep):** Manual QA matrix execution on Windows and documentation overhaul.

Core Academic Bibliography & Standards

- Diffie & Hellman (1976):** New Directions in Cryptography. IEEE Transactions on Information Theory. [\[IEEE\]](#)
- McGrew & Viega (2004):** Security of Galois/Counter Mode (GCM). INDOCRYPT / NIST SP 800-38D. [\[Springer\]](#)
- Krawczyk (2010):** Cryptographic Extraction & HKDF. CRYPTO 2010 / RFC 5869. [\[IACR\]](#)
- Unger et al. (2015):** SoK: Secure Messaging. IEEE Symposium on Security and Privacy (S&P). [\[IEEE S&P\]](#)
- Cohn-Gordon et al. (2017):** Formal Analysis of Signal Protocol. IEEE EuroS&P. [\[IACR\]](#)
- Vaudenay (2005):** Secure Communications Based on SAS. CRYPTO 2005. [\[IACR\]](#)
- Frohoff & Lawrence (2015):** Deserializing Objects Exploit. AppSecCali 2015. [\[AppSecCali\]](#)
- IETF Standards:** RFC 3526 (DH-14), RFC 5869 (HKDF), RFC 8446 (TLS 1.3), RFC 9420 (MLS). [\[IETF RFCs\]](#)